

SOFTWARE PARA LA GESTIÓN DE PRÁCTICAS ACADÉMICAS

Propuesta de Proyecto Integrador - Primer Avance

Aplicación web con arquitectura REST, PostgreSQL y MongoDB

Dato	Información
Programa	Ingeniería de Sistemas
Período académico	II - 2026
Semestre	Quinto
Cursos vinculados	Ingeniería del Software II · Programación III · Teoría General de Sistemas
Equipo	Jesus Daniel Rueda Castillo, Alonso Andres Henao Roa y Rafael Fabian Carreño Barrera
Institución	Universidad de Investigación y Desarrollo - UDI
Fecha de entrega del primer avance	30 de agosto de 2026

Propósito de este documento

Actualizar la propuesta y el prototipo existente para implementar una solución web coherente, utilizando Figma para el diseño y validación de los prototipos de interfaz, separando la interfaz funcional, la API REST y la persistencia, y utilizando PostgreSQL como base transaccional y MongoDB como repositorio NoSQL para datos semiestructurados de trazabilidad y eventos.

Contenido

1. Introducción
2. Descripción del problema
3. Objetivos
4. Justificación
5. Propuesta del plan del proyecto y arquitectura
6. Análisis de requerimientos del software
7. Diseño UML
8. Modelado de la base de datos
9. Prototipo de mediana fidelidad
10. Referencias bibliográficas
11. Anexos técnicos

Documento base: formulación oficial del Proyecto Integrador 2026-II y prototipo de alta fidelidad existente del sistema.

1. Introducción

Las prácticas académicas constituyen un proceso crítico en los programas de formación porque conectan al estudiante con escenarios reales de desempeño y generan información que debe ser administrada por múltiples actores: dirección de programa, coordinación de prácticas, docentes asesores, estudiantes e instituciones receptoras. Cuando este proceso se gestiona mediante hojas de cálculo, correos y archivos aislados, la institución pierde trazabilidad, aumenta el trabajo manual y dificulta la consolidación de evidencias e indicadores.

El proyecto propone construir un Software para la Gestión de Prácticas Académicas como aplicación web. La nueva implementación aprovechará el prototipo visual y el análisis funcional ya desarrollado, pero reemplazará la orientación anterior de aplicación de escritorio y base de datos Oracle por una arquitectura cliente-servidor basada en una interfaz web, una API REST y persistencia polígota.

PostgreSQL se utilizará como sistema gestor relacional para la información transaccional y estructurada, especialmente usuarios, programas, cursos, prácticas, grupos, matrículas, asignaciones y evaluaciones. MongoDB se incorporará como componente NoSQL para información semiestructurada que puede evolucionar en el tiempo, como eventos de auditoría, historial de cambios y notificaciones. Esta separación evita utilizar NoSQL de forma artificial y permite justificar técnicamente la responsabilidad de cada motor de persistencia.

La solución se desarrollará de forma incremental, priorizando primero los procesos esenciales: autenticación, usuarios y roles, programas, instituciones, prácticas, grupos y asignaciones. En iteraciones posteriores se incorporarán actividades, evidencias, control de horas, evaluaciones, reportes e indicadores. De esta manera se busca cumplir los avances definidos para el Proyecto Integrador y mantener un producto demostrable durante todo el semestre.

Decisión de arquitectura

El navegador nunca se conectará directamente a PostgreSQL ni a MongoDB. Toda interacción se realizará a través de la API REST, donde se aplicarán autenticación, autorización, validaciones y reglas de negocio.

2. Descripción del problema

2.1 Situación actual

La gestión de las prácticas involucra información que cambia continuamente: estudiantes activos, instituciones receptoras, convenios, grupos, docentes asignados, horas cumplidas, evidencias, evaluaciones y reportes. Cuando cada actor conserva su propia copia de los datos, es difícil determinar cuál es la información vigente y verificar quién realizó una modificación.

El Director del Programa necesita una visión consolidada para supervisar el cumplimiento de horas, conocer el estado de las prácticas, consultar evaluaciones y producir reportes. El Coordinador necesita organizar grupos y asignaciones. Los docentes asesores requieren registrar seguimiento y evaluación, mientras que estudiantes e instituciones deben participar en el registro y validación de actividades y horas. La ausencia de un sistema centralizado obliga a realizar verificaciones manuales y hace más lenta la preparación de informes.

El prototipo existente demuestra que los procesos pueden representarse mediante módulos diferenciados y paneles por rol. Sin embargo, la documentación previa combina elementos de una aplicación de escritorio con diagramas que ya sugieren una solución web REST. Por ello, el nuevo desarrollo debe unificar desde el inicio la arquitectura y el modelo de datos para que la documentación, los diagramas y el código representen el mismo sistema.

2.2 Problema central

¿Cómo diseñar e implementar una aplicación web que centralice la gestión de prácticas académicas, garantice acceso diferenciado por roles, preserve la integridad de la información, permita trazabilidad de las acciones y facilite el seguimiento de estudiantes, docentes, instituciones y prácticas mediante una arquitectura REST con PostgreSQL y MongoDB?

2.3 Alcance del problema a resolver

- Fragmentación de información entre diferentes herramientas y responsables.
- Dificultad para conocer en tiempo real el estado de estudiantes, grupos y prácticas.
- Falta de trazabilidad sobre cambios y operaciones críticas.
- Control manual de horas reglamentarias y evidencias.
- Dificultad para distribuir y consultar la carga de docentes asesores.
- Generación manual o repetitiva de reportes e indicadores.
- Riesgo de inconsistencias entre la información reportada por estudiantes, docentes e instituciones.
- Necesidad de contar con una solución web accesible desde navegador y documentada mediante una API REST.

3. Objetivos

3.1 Objetivo general

Diseñar e implementar una aplicación web para la gestión de prácticas académicas que centralice el registro, seguimiento, evaluación y reporte de los procesos asociados, mediante una arquitectura REST con autenticación por roles, PostgreSQL para la información transaccional y MongoDB para datos semiestructurados de trazabilidad, notificaciones e historial de eventos.

3.2 Objetivos específicos

1. Analizar y depurar los requerimientos del prototipo existente, identificando actores, procesos, reglas de negocio y prioridades de implementación.
2. Diseñar una arquitectura web cliente-servidor en la que el frontend consuma una API REST y la lógica de negocio permanezca desacoplada de las tecnologías de persistencia.
3. Modelar la base de datos relacional en PostgreSQL para usuarios, programas, cursos, prácticas, instituciones, grupos, matrículas, asignaciones, actividades, horas, rúbricas y evaluaciones.
4. Diseñar colecciones MongoDB para auditoría, historial de cambios y notificaciones, definiendo claramente la frontera entre datos SQL y NoSQL.
5. Implementar autenticación y autorización basada en roles mediante tokens JWT, evitando que el usuario seleccione manualmente su rol al iniciar sesión.
6. Desarrollar los módulos prioritarios del sistema y exponer sus operaciones mediante endpoints REST documentados con OpenAPI.
7. Implementar operaciones CRUD, validaciones y reglas de negocio para los casos de uso priorizados en cada avance del Proyecto Integrador.
8. Incorporar mecanismos de auditoría, manejo de errores, pruebas y documentación técnica que permitan mantener y desplegar la aplicación de forma reproducible.
9. Generar reportes e indicadores que apoyen el seguimiento académico y la toma de decisiones por parte de la dirección y coordinación del programa.

4. Justificación

La implementación de una plataforma web permite que los diferentes actores consulten y actualicen la información desde un navegador sin depender de una instalación local específica. Este enfoque es especialmente útil en un proceso donde intervienen usuarios con responsabilidades distintas y donde la información debe mantenerse centralizada.

El uso de PostgreSQL se justifica porque gran parte del dominio presenta relaciones claras y reglas de integridad: un estudiante pertenece a una matrícula, una matrícula corresponde a un grupo, un grupo se relaciona con una práctica y una institución, y una evaluación debe conservar referencias consistentes a estudiante, rúbrica y evaluador. Estas características requieren transacciones y restricciones relacionales.

MongoDB se incorpora para información cuyo esquema puede evolucionar y cuyo valor principal es conservar el contexto del evento más que establecer relaciones complejas. Las auditorías, notificaciones y snapshots de cambios pueden contener propiedades diferentes según la acción ejecutada. Al conservar los identificadores de las entidades relacionales se obtiene trazabilidad sin duplicar la responsabilidad del modelo transaccional.

La arquitectura REST también facilita la evaluación académica del proyecto porque permite demostrar de manera independiente el frontend, la lógica de negocio, las operaciones CRUD, las reglas de seguridad y las consultas a las bases de datos. Además, deja una base técnica que puede evolucionar hacia aplicaciones móviles u otros clientes sin reescribir la lógica central.

5. Propuesta del plan del proyecto y arquitectura

5.1 Alcance funcional propuesto

- Autenticación y autorización por roles.
- Gestión de usuarios y perfiles.
- Gestión de programas académicos y cursos.
- Gestión de instituciones receptoras y convenios.
- Gestión de prácticas académicas y grupos.
- Asignación de docentes y estudiantes.
- Registro de actividades, evidencias y horas.
- Confirmación de horas por la institución receptora.
- Configuración de rúbricas y evaluación de estudiantes.
- Consulta de retroalimentación y avance por estudiante.
- Dashboard, reportes e indicadores.
- Auditoría de operaciones críticas y notificaciones.

5.2 Enfoque de desarrollo

Se propone un desarrollo incremental e iterativo. Durante la fase de diseño, las interfaces y los flujos de interacción se definirán y validarán en Figma antes de iniciar su implementación funcional. Cada incremento debe producir una versión ejecutable y demostrable. El equipo puede trabajar con sprints cortos de una a dos semanas, un tablero Kanban para el seguimiento y una rama principal protegida en GitHub. Cada caso de uso se considera terminado cuando cuenta con interfaz implementada, endpoint REST, validaciones, persistencia, manejo de errores y evidencia de prueba.

Fase	Enfoque	Resultado esperado
Fase 1	Propuesta y diseño	Requerimientos, arquitectura, UML, modelo de datos y prototipo de mediana fidelidad en Figma.
Fase 2	Núcleo funcional	Prototipo de alta fidelidad en Figma y núcleo funcional con login, roles, usuarios, programas, instituciones, prácticas, grupos y CRUD básicos.
Fase 3	Operación de prácticas	Asignaciones, actividades, evidencias, horas y validaciones de negocio.
Fase 4	Evaluación y analítica	Rúbricas, evaluaciones, reportes, dashboard, auditoría y notificaciones.
Fase 5	Cierre	Pruebas, correcciones, documentación, release, manuales, video y sustentación.

5.3 Arquitectura lógica

La solución se organiza en capas. Figma se utilizará exclusivamente para el diseño y prototipado de las interfaces, por lo que no forma parte de la arquitectura de ejecución del sistema. La aplicación funcional tendrá un frontend web que implementará las vistas validadas en Figma y consumirá contratos HTTP. La API REST concentrará autenticación, autorización, validación y reglas del dominio. La capa de persistencia contendrá repositorios independientes para PostgreSQL y MongoDB.

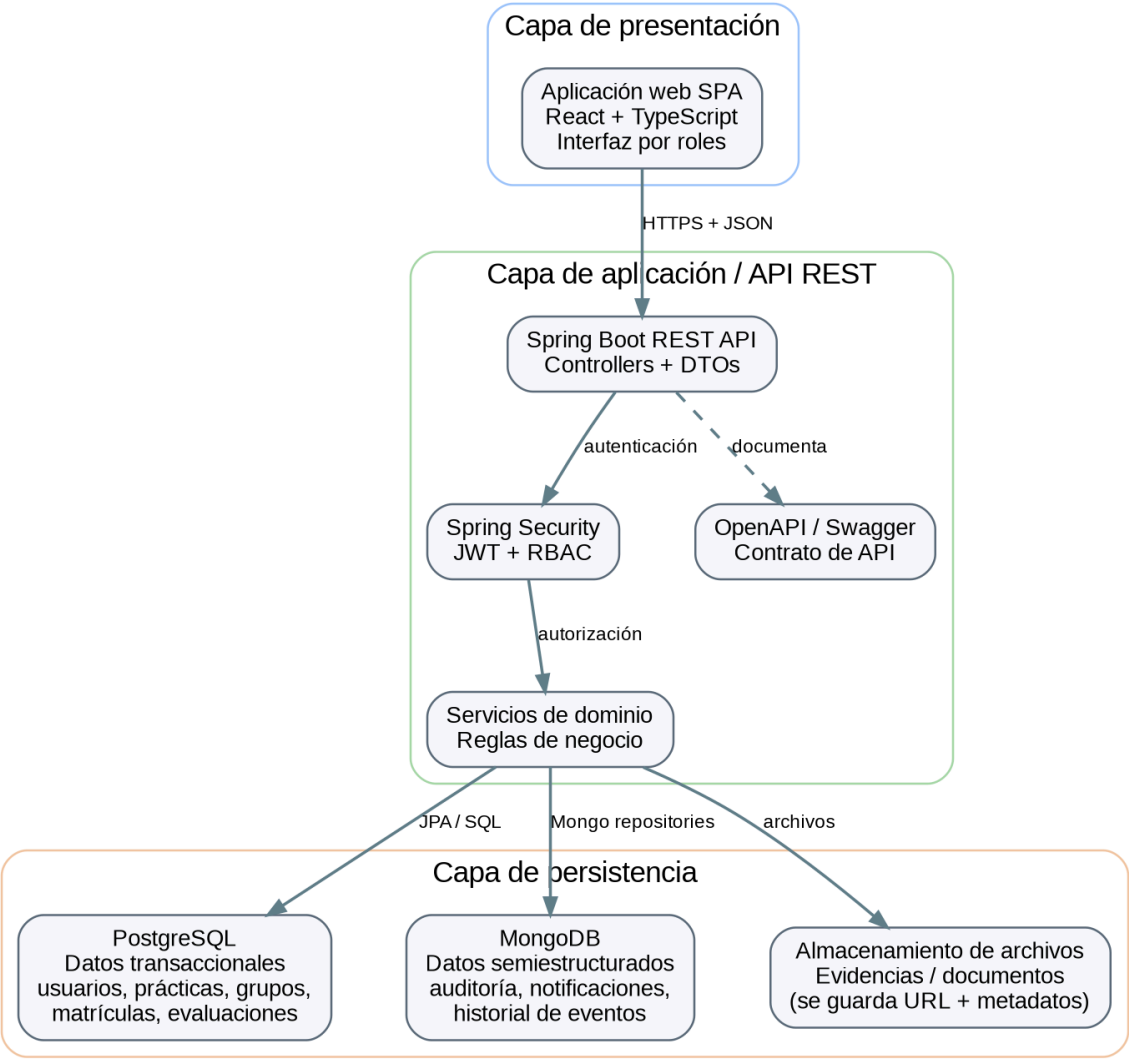


Figura 1. Arquitectura propuesta de la aplicación web.

Componente	Tecnología propuesta	Responsabilidad
Prototipado	Figma	Diseño de prototipos de mediana y alta fidelidad, navegación, flujos y componentes visuales antes de programar el frontend.
Frontend	React + TypeScript	SPA web, componentes, rutas y formularios por rol.
Backend	Spring Boot	API REST, servicios de dominio y acceso a datos.
Seguridad	Spring Security + JWT	Autenticación, autorización RBAC y protección de endpoints.
Base SQL	PostgreSQL	Fuente principal de verdad para datos transaccionales.
Base NoSQL	MongoDB	Auditoría, notificaciones e historial de eventos.

Componente	Tecnología propuesta	Responsabilidad
Documentación	OpenAPI / Swagger	Contrato y prueba interactiva de endpoints REST.
Pruebas	JUnit / Mockito / Postman	Pruebas de servicios, controladores y API.
Versionamiento	Git + GitHub	Control de cambios, issues, ramas y releases.
Ejecución	Docker Compose (recomendado)	Entorno reproducible para API, PostgreSQL y MongoDB.

5.4 Persistencia polígota: PostgreSQL + MongoDB

PostgreSQL será la fuente de verdad del estado académico. MongoDB no reemplaza relaciones ni transacciones del sistema; se utiliza como repositorio complementario para información semiestructurada.

Componente	Datos	Justificación
PostgreSQL	usuarios, roles, programas, cursos, prácticas, instituciones, convenios, grupos, matrículas, asignaciones, horas, actividades, rúbricas, evaluaciones	Integridad referencial, transacciones, consultas relacionales, restricciones y consistencia.
MongoDB	auditoria_eventos, historial_cambios, notificaciones	Documentos con estructura variable, contexto completo del evento y evolución del esquema.
Almacenamiento de archivos	PDF, imágenes y documentos cargados como evidencia	El archivo se conserva en almacenamiento de archivos; PostgreSQL guarda la URL y metadatos mínimos. No se recomienda almacenar binarios grandes dentro de las tablas transaccionales.

Regla de diseño

No se crearán relaciones de negocio críticas que dependan exclusivamente de un documento MongoDB. Cuando un evento de MongoDB se refiera a un usuario, práctica, grupo o matrícula, conservará el identificador de PostgreSQL, pero el estado oficial de la entidad seguirá residiendo en PostgreSQL.

5.5 Seguridad

- Contraseñas almacenadas mediante hash seguro; nunca en texto plano.
- Inicio de sesión con correo y contraseña. El rol se obtiene desde el servidor; no se selecciona en el formulario de acceso.
- JWT con fecha de expiración y claims mínimos necesarios.
- Autorización por rol y, cuando aplique, por pertenencia al recurso solicitado.
- Validación de datos en frontend y backend; la validación del backend es obligatoria.
- Manejo centralizado de errores para evitar exposición de información técnica sensible.
- Registro de operaciones críticas en la colección de auditoría.
- HTTPS para ambientes desplegados y variables de entorno para secretos de conexión.

5.6 Cronograma alineado con las entregas del Proyecto Integrador

Periodo	Hito	Trabajo principal
Hasta 30 ago. 2026	Primer avance	Propuesta, arquitectura, requerimientos, UML, MER y prototipo de mediana fidelidad en Figma.

Periodo	Hito	Trabajo principal
31 ago. - 20 sep.	Sprint núcleo	Login, roles, usuarios, programas, instituciones y prácticas.
21 sep. - 11 oct.	Segundo avance	Prototipo de alta fidelidad en Figma, grupos, asignaciones y CRUD demostrable; objetivo de cubrir al menos 50 % de casos de uso priorizados.
12 oct. - 01 nov.	Sprint operación	Actividades, evidencias, horas, convenios y evaluación.
02 nov. - 15 nov.	Entrega final	Reportes, auditoría, pruebas, release y documentación.
Última semana de nov.	Sustentación	Video, demostración, artículo y póster.

6. Análisis de requerimientos del software

6.1 Actores

Actor	Responsabilidad principal
Director del Programa	Administra usuarios, programas, prácticas, consulta indicadores y genera reportes.
Coordinador de Práctica	Gestiona grupos, instituciones, convenios, asignaciones y seguimiento operativo.
Docente Asesor	Consulta estudiantes asignados, realiza seguimiento y evaluaciones.
Estudiante	Consulta su práctica, registra actividades, carga evidencias y consulta retroalimentación.
Institución Receptora	Consulta estudiantes asignados, confirma horas y verifica información del convenio.
Sistema	Genera auditorías, notificaciones, validaciones automáticas e indicadores.

6.2 Requerimientos funcionales

ID	Requerimiento	Descripción	Actor	Prioridad
RF-01	Autenticar usuarios	El sistema debe permitir iniciar sesión mediante correo y contraseña y emitir un token de sesión válido.	Todos	Alta
RF-02	Autorizar por rol	El sistema debe habilitar o bloquear operaciones de acuerdo con el rol y permisos del usuario autenticado.	Sistema	Alta
RF-03	Gestionar usuarios	Crear, consultar, actualizar, activar e inactivar usuarios.	Director	Alta
RF-04	Gestionar programas	Crear, consultar y actualizar programas académicos.	Director	Alta
RF-05	Gestionar cursos	Administrar cursos asociados a un programa.	Director	Media
RF-06	Gestionar prácticas	Crear, consultar, actualizar y cambiar el estado de prácticas académicas.	Director / Coordinador	Alta
RF-07	Gestionar instituciones	Registrar, consultar y actualizar instituciones receptoras.	Director / Coordinador	Alta

ID	Requerimiento	Descripción	Actor	Prioridad
RF-08	Gestionar convenios	Registrar vigencia, estado y datos básicos de convenios institucionales.	Coordinador	Alta
RF-09	Gestionar grupos	Crear grupos por práctica, institución, fechas y cupo máximo.	Coordinador	Alta
RF-10	Asignar docente	Asignar o reasignar un docente asesor a un grupo.	Coordinador	Alta
RF-11	Asignar estudiantes	Vincular uno o varios estudiantes a un grupo con validación de cupo y duplicidad.	Coordinador	Alta
RF-12	Consultar estudiantes asignados	Permitir a coordinación, docente e institución consultar estudiantes según su ámbito de responsabilidad.	Varios	Alta
RF-13	Registrar actividades	El estudiante debe poder registrar actividades asociadas a su matrícula de práctica.	Estudiante	Alta
RF-14	Cargar evidencias	Permitir adjuntar evidencias a una actividad validando tipo, tamaño y pertenencia.	Estudiante	Media
RF-15	Registrar horas	Permitir registrar horas ejecutadas asociadas a la matrícula.	Estudiante / Institución	Alta
RF-16	Confirmar horas	La institución receptora debe poder validar las horas reportadas dentro de los límites de la práctica.	Institución	Alta
RF-17	Gestionar rúbricas	Crear rúbricas con criterios, ponderaciones y niveles de desempeño.	Director / Docente	Media
RF-18	Evaluar estudiante	Registrar evaluación, niveles alcanzados, nota y observación.	Docente	Alta
RF-19	Retroalimentar	Registrar comentarios individuales sobre desempeño y entregas.	Docente	Alta
RF-20	Consultar avance	Mostrar al estudiante horas, actividades, evidencias y evaluación disponible.	Estudiante	Alta
RF-21	Generar reportes	Generar reportes por práctica, grupo, institución, estudiante y periodo.	Director / Coordinador	Alta
RF-22	Mostrar dashboard	Presentar indicadores resumidos del estado de prácticas y estudiantes.	Director	Media
RF-23	Generar notificaciones	Crear avisos por eventos relevantes como déficit de horas, cambios de asignación o vencimiento de convenio.	Sistema	Media
RF-24	Registrar auditoría	Conservar un registro de operaciones críticas indicando usuario, acción, entidad, fecha y contexto.	Sistema	Alta

6.3 Requerimientos no funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	Las contraseñas deben almacenarse con hash seguro y nunca exponerse en respuestas o reportes.
RNF-02	Autorización	Todos los endpoints protegidos deben verificar rol y permisos antes de ejecutar la operación.
RNF-03	Rendimiento	Las consultas frecuentes deberán responder en menos de 3 segundos bajo el escenario de carga definido para las pruebas académicas.
RNF-04	Usabilidad	La interfaz debe ser consistente, responsive y comprensible para usuarios sin capacitación técnica avanzada.
RNF-05	Integridad	PostgreSQL debe aplicar claves primarias, foráneas, restricciones únicas y reglas de consistencia.
RNF-06	Trazabilidad	Las operaciones críticas deben producir un evento de auditoría consultable.
RNF-07	Privacidad	El sistema debe limitar la exposición de datos personales a los usuarios autorizados y aplicar principios de minimización.
RNF-08	Mantenibilidad	El código debe organizarse por dominios o módulos, con separación entre controladores, servicios, DTOs y repositorios.
RNF-09	Documentación	La API debe estar documentada con OpenAPI y el repositorio debe incluir instrucciones de ejecución.
RNF-10	Portabilidad	El entorno de desarrollo debe poder levantarse de forma reproducible; se recomienda Docker Compose para las bases de datos.
RNF-11	Respaldo	Debe existir un procedimiento documentado de respaldo de PostgreSQL y MongoDB para el entorno de demostración.
RNF-12	Manejo de errores	La API debe responder con códigos HTTP coherentes y mensajes controlados sin filtrar trazas internas.

6.4 Reglas de negocio principales

- Un estudiante no puede tener dos matrículas activas en el mismo grupo y período.
- No se puede superar el cupo máximo del grupo.
- Una institución debe tener convenio vigente para recibir nuevas asignaciones.
- Un grupo solo puede pertenecer a una práctica y a una institución en una vigencia determinada.
- Las horas confirmadas no pueden superar el total permitido sin una operación de ajuste autorizada y auditada.
- Una evaluación debe estar asociada a una matrícula activa o finalizada y a una rúbrica válida.
- Los cambios críticos de estado deben conservar trazabilidad.
- Las eliminaciones funcionales se realizarán preferiblemente mediante inactivación o soft delete cuando exista historial dependiente.

7. Diseño UML

7.1 Diagrama de casos de uso

El siguiente diagrama resume los principales objetivos de interacción de cada actor. Los casos de uso detallados se derivarán del backlog y se acompañarán con flujos alternos durante el segundo avance.

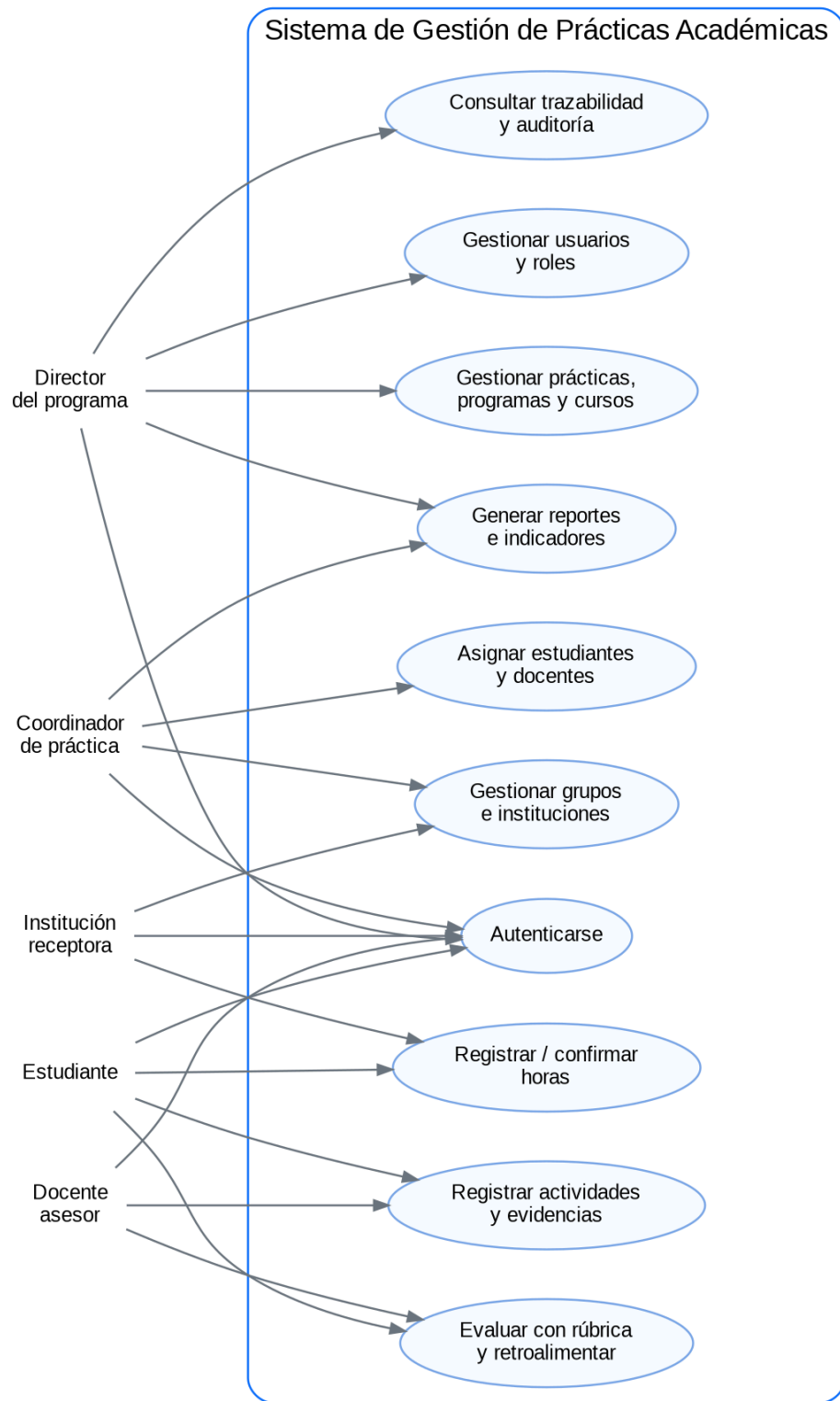


Figura 2. Casos de uso principales por actor.

Código	Caso de uso	Actor	Resultado esperado
CU-01	Autenticarse	Todos	Acceder al sistema y recibir permisos según el rol almacenado.
CU-02	Gestionar práctica	Director / Coordinador	Crear, consultar, actualizar y controlar el estado de una práctica.
CU-03	Crear grupo	Coordinador	Definir institución, fechas y cupo para ejecutar una práctica.
CU-04	Asignar estudiantes	Coordinador	Vincular estudiantes disponibles a un grupo.
CU-05	Asignar docente	Coordinador	Vincular un docente asesor al grupo.
CU-06	Registrar actividad y evidencia	Estudiante	Documentar el trabajo realizado durante la práctica.
CU-07	Confirmar horas	Institución	Validar horas reportadas por estudiantes asignados.
CU-08	Evaluar estudiante	Docente	Aplicar una rúbrica y registrar retroalimentación.
CU-09	Generar reportes	Director / Coordinador	Obtener información consolidada para seguimiento.
CU-10	Auditar operaciones	Director / Sistema	Consultar trazabilidad de operaciones críticas.

7.2 Diagrama de dominio

El modelo de dominio representa las entidades de negocio más relevantes sin depender todavía de detalles de implementación. Sirve como puente entre los casos de uso y el modelo de base de datos.

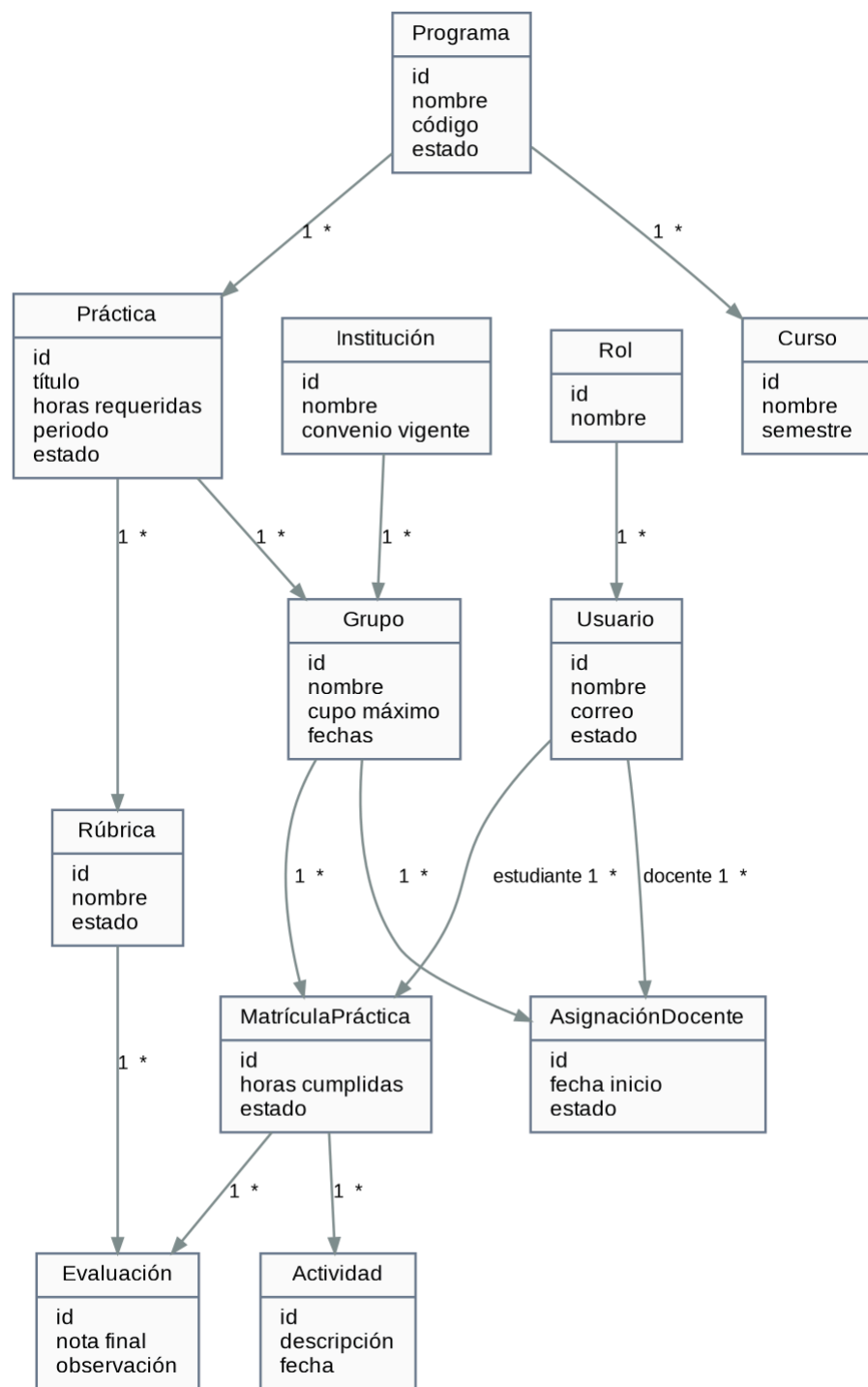


Figura 3. Modelo de dominio simplificado.

8. Modelado de la base de datos

8.1 Principios del modelo

- PostgreSQL es la fuente de verdad del estado operativo y académico.

8.2 Modelo entidad-relación relacional propuesto

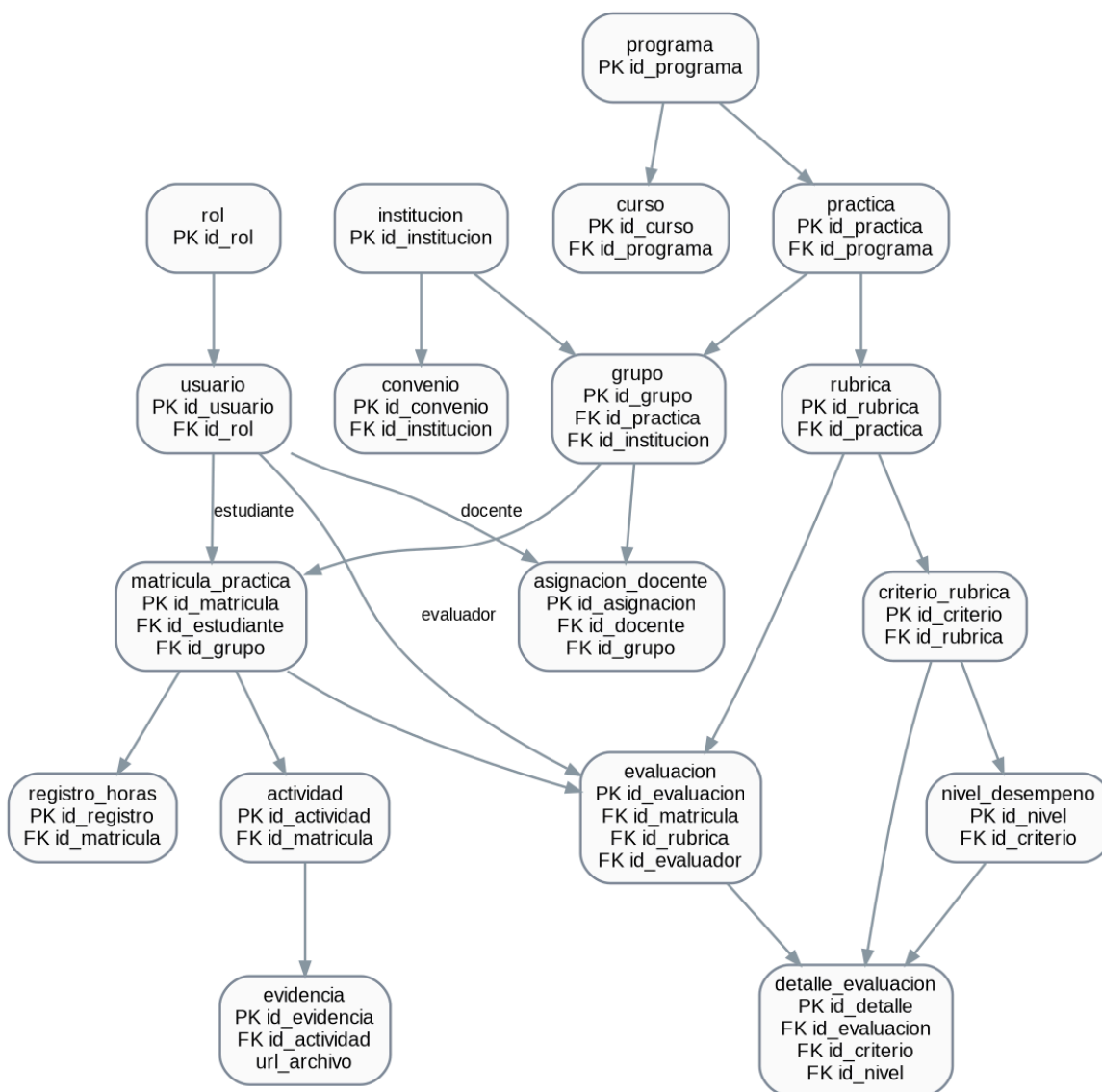


Figura 4. Modelo entidad-relación propuesto para PostgreSQL.

Tabla PostgreSQL	Responsabilidad
rol	Catálogo de roles autorizados.
usuario	Credenciales, identidad básica y estado.

Tabla PostgreSQL	Responsabilidad
programa	Programa académico propietario de cursos y prácticas.
curso	Curso asociado al programa.
practica	Definición de la práctica, horas, periodo y estado.
institucion	Institución receptora.
convenio	Vigencia y estado del convenio de una institución.
grupo	Ejecución de una práctica en una institución con fechas y cupo.
matricula_practica	Vinculación de estudiante con grupo y avance de horas.
asignacion_docente	Vinculación del docente asesor con un grupo.
actividad	Actividad registrada por el estudiante.
registro_horas	Detalle de horas reportadas / confirmadas.
evidencia	Metadatos y URL del archivo asociado a una actividad.
rubrica	Instrumento de evaluación relacionado con una práctica.
criterio_rubrica	Criterios y ponderaciones.
nivel_desempeno	Niveles posibles para cada criterio.
evaluacion	Evaluación general del estudiante.
detalle_evaluacion	Nivel seleccionado por criterio dentro de una evaluación.

8.3 Colecciones MongoDB propuestas

Colección	Contenido	Uso
auditoria_eventos	Registra quién realizó una acción, qué entidad fue afectada, fecha, IP/contexto y datos relevantes del cambio.	Trazabilidad y auditoría.
historial_cambios	Conserva snapshots parciales antes/después para operaciones seleccionadas.	Explicar cambios sin rigidizar el esquema.
notificaciones	Mensajes generados por eventos del sistema con destinatarios, estado de lectura y datos contextuales.	Estructura variable según tipo de notificación.

8.4 Ejemplos de documentos NoSQL

Evento de auditoría

```
{
  "usuarioId": 25,
  "rol": "COORDINADOR",
  "accion": "ASIGNACION_ESTUDIANTE",
  "entidad": "matricula_practica",
  "entidadId": 441,
  "fecha": "2026-08-19T21:20:13-05:00",
  "contexto": {"estudianteId": 154, "grupoId": 12, "institucionId": 4}
}
```

Notificación

```
{
  "tipo": "CONVENIO_POR_VENCER",
  "destinatarios": [8, 12],
  "fechaCreacion": "2026-10-01T08:00:00-05:00",
  "leidaPor": [],
  "datos": {"institucionId": 4, "diasRestantes": 30}
}
```




- En el prototipo de Figma, eliminar la selección manual del rol en el inicio de sesión; en la aplicación funcional el servidor debe obtener el rol desde el usuario autenticado.

- Unificar los nombres de estados de prácticas, grupos, matrículas y usuarios para que coincidan con el modelo de datos.
- Definir claramente qué acciones son visibles para Director, Coordinador, Docente, Estudiante e Institución.
- Agregar estados de carga, error, vacío y confirmación para las operaciones asíncronas del frontend.
- Evitar eliminar físicamente registros con dependencias históricas; mostrar acciones de activar/inactivar cuando corresponda.
- Definir en Figma componentes reutilizables para tablas, filtros, modales, alertas, paginación y formularios, manteniendo consistencia visual entre los diferentes roles.
- Alinear cada pantalla con un endpoint o conjunto de endpoints REST documentados.

10. Referencias bibliográficas

Beck, K. (2002). Test Driven Development: By Example. Addison-Wesley.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine.

Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Larman, C. (2003). UML y patrones: Una introducción al análisis y diseño orientado a objetos (2.ª ed.). Pearson Educación.

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). Fundamentos de bases de datos (6.ª ed.). McGraw-Hill.

Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson Educación.

Object Management Group. (2017). Unified Modeling Language (UML), Version 2.5.1.

OpenAPI Initiative. OpenAPI Specification.

PostgreSQL Global Development Group. PostgreSQL Documentation.

MongoDB, Inc. MongoDB Documentation.

OWASP Foundation. OWASP Application Security Verification Standard y OWASP Top 10.

11. Anexos técnicos

Anexo A. Borrador de endpoints REST

Método	Endpoint	Propósito
POST	/api/v1/auth/login	Autenticación.
GET	/api/v1/usuarios	Listar usuarios autorizados.
POST	/api/v1/usuarios	Crear usuario.
GET	/api/v1/programas	Listar programas.
POST	/api/v1/practicas	Crear práctica.
GET	/api/v1/practicas	Listar / filtrar prácticas.
PUT	/api/v1/practicas/{id}	Actualizar práctica.

Método	Endpoint	Propósito
POST	/api/v1/grupos	Crear grupo.
GET	/api/v1/grupos/{id}/estudiantes	Consultar estudiantes del grupo.
POST	/api/v1/grupos/{id}/estudiantes	Asignar estudiantes.
POST	/api/v1/grupos/{id}/docente	Asignar docente.
POST	/api/v1/matriculas/{id}/actividades	Registrar actividad.
POST	/api/v1/actividades/{id}/evidencias	Cargar evidencia.
POST	/api/v1/matriculas/{id}/horas	Registrar horas.
PATCH	/api/v1/horas/{id}/confirmacion	Confirmar / rechazar horas.
POST	/api/v1/evaluaciones	Registrar evaluación.
GET	/api/v1/reportes/practicas/{id}	Generar reporte de práctica.
GET	/api/v1/auditoria	Consultar eventos autorizados.

Anexo B. Estructura propuesta del repositorio

```

gestion-practicas-academicas/
├── frontend/
│   └── src/
│       ├── api/
│       ├── components/
│       ├── layouts/
│       ├── pages/
│       ├── router/
│       └── types/
├── backend/
│   └── src/main/java/...
│       ├── auth/
│       ├── usuarios/
│       ├── practicas/
│       ├── grupos/
│       ├── matriculas/
│       ├── evaluaciones/
│       ├── reportes/
│       └── auditoria/
├── database/
│   ├── postgresql/
│   └── mongodb/
├── docs/
├── docker-compose.yml
└── README.md
  
```

Anexo C. Priorización para el segundo avance

Prioridad	Componente	Razón
P0	Login y autorización por roles	Debe estar listo primero porque protege el resto del sistema.
P0	Usuarios, programas, instituciones y prácticas	Catálogos y entidades base.
P0	Grupos y asignación de estudiantes	Flujo central del sistema y demostración del valor del producto.
P0	CRUD PostgreSQL	Evidencia directa de la integración de base de datos.
P1	Asignación docente y consultas por rol	Completa el flujo de coordinación.
P1	Auditoría MongoDB	Demuestra uso real del componente NoSQL.
P2	Actividades, evidencias y horas	Siguiente incremento funcional.
P2	Rúbricas, evaluación, reportes y dashboard	Cierre de alcance para la versión final.

Anexo D. Criterios mínimos de aceptación por historia / caso de uso

- La interfaz correspondiente puede ejecutarse desde el frontend.

- El endpoint REST está implementado y documentado.
- El backend valida datos obligatorios y reglas de negocio.
- La operación persiste o consulta correctamente la fuente de datos definida.
- Los errores se convierten en respuestas HTTP controladas.
- La operación respeta los permisos del rol autenticado.
- Existe al menos una prueba o evidencia reproducible de funcionamiento.
- Las operaciones críticas generan el evento de auditoría definido.

Estado del documento

Versión ajustada para el primer avance. El prototipado de la interfaz se realizará en Figma; la implementación funcional será independiente y seguirá la arquitectura web REST definida con PostgreSQL y MongoDB. Antes de entregar, se deben incorporar las exportaciones definitivas del prototipo de mediana fidelidad elaborado en Figma y verificar la notación UML exigida por la asignatura.